

Week 5 Lab — Your first tool-calling agent

CIS 4394 Agentic AI · Work in pairs · One laptop is enough · Nothing here costs money and nothing leaves your computer

Two roles — swap after Part B.

Driver types the commands. **Recorder** fills in the observation tables and gets ready to report to the class.

If your laptop will not cooperate for more than ~5 minutes, stop debugging — use your partner's, or raise your hand and join a nearby pair. The point of today is what you observe, not your Python setup.

Part A — Setup (about 5 minutes)

A1 · Is Ollama running? (same on both systems)

```
ollama list
```

You should see `qwen2.5:0.5b` (the model you downloaded last week) in the list. **Write down its exact name** — you will need it later, including the part after the colon.

Error, or an empty list? Open the Ollama app first (Launchpad on Mac, Start menu on Windows), wait for its icon to appear, then run the command again.

A2 · Do you have Python?

MAC — USE TERMINAL

```
python3 --version
```

Expect `Python 3.9` or higher. If macOS offers to install developer tools, accept and wait.

WINDOWS — USE POWERSHELL

```
py -3 --version
```

Expect `Python 3.9` or higher. If the command is not found, install Python from the Microsoft Store, then reopen PowerShell.

A3 · Install the one package we need

MAC

```
python3 -m pip install ollama
```

WINDOWS

```
py -3 -m pip install ollama
```

"But I already installed Ollama last week." Two different things with the same name. The **Ollama app** is the engine that loads the model and runs it on your computer. The **ollama Python package** is a small library that lets your Python code send requests to that engine. You need both — the app to run the model, the package so your code can talk to it.

This is a small download and should take seconds. It is the only package today.

A4 · Download the starter file

MAC

```
cd ~/Desktop
curl -O https://sherryfu0315.github.io/cis4394-agentic-ai/materials/week5_ollama_starter/1_agent_raw.py
```

WINDOWS

```
cd ~\Desktop
curl.exe -O https://sherryfu0315.github.io/cis4394-agentic-ai/materials/week5_ollama_starter/1_agent_raw.py
```

The file lands on your Desktop. You can also just download it in a browser from the same address and drag it to the Desktop.

Part B — Run it (the simplest possible question)

MAC

```
python3 1_agent_raw.py "What is 4817 * 293?"
```

WINDOWS

```
py -3 1_agent_raw.py "What is 4817 * 293?"
```

After a second or two you should see something like this:

```
[step 1] model requests -> calculator({'expression': '4817 * 293'})  
harness ran it    <- 1411381
```

```
[step 2] FINAL ANSWER:  
The result of 4817 * 293 is 1411381.
```

Look carefully at the two arrows. They point in opposite directions, and that is the whole lesson of today. One line is the model *asking* for something. The other is a piece of ordinary Python code *doing* it.

Recorder — fill this in

QUESTION	YOUR ANSWER
Which tool did the model ask for?	
What arguments did it send?	
What value came back?	
Did the model multiply those numbers, or did something else? How can you tell from the output?	

Now try a question that needs no tool

MAC

```
python3 1_agent_raw.py "Say hello in one sentence."
```

WINDOWS

```
py -3 1_agent_raw.py "Say hello in one sentence."
```

QUESTION	YOUR ANSWER
Did it call a tool this time?	
Who decided that — the model, or the code?	

Part C — Finished early? Make it fail (about 5 minutes)

Run it with **no question at all** — the file then uses a harder, multi-step question about an invoice:

MAC

```
python3 1_agent_raw.py
```

WINDOWS

```
py -3 1_agent_raw.py
```

The question is: *an invoice of 4,817 units at €293 each, plus 18% tax — what is the total in US dollars?* The correct answer is **\$1,798,663.95**.

QUESTION	YOUR ANSWER
Did your model get it right?	
Which tools did it call, in what order?	
If it was wrong — was the mistake in calling the tools, or in deciding what steps to take ?	
Run it twice more. Same answer every time?	

Expect failure here, and do not fix it. A 0.5B model is about the smallest useful model that exists. Watching it fail on a multi-step task — while calling its tools perfectly — is the point of this part.

Bring these to the class discussion

- 1 · When the model "called the calculator," what did it actually produce — and who ran the code?
- 2 · Your agent can only do what its tools let it do. If you added a tool called `issue_refund`, what new thing could go wrong that cannot go wrong today?
- 3 · If you ran the same question three times and got different answers, what does that mean for a company that wants to put this in front of customers?

Each pair should be ready to give a one-sentence answer to at least one of these. Recorders, bring your tables.

If something goes wrong

WHAT YOU SEE	WHAT TO DO
No module named 'ollama'	Part A3 did not finish, or you are running a different Python. Mac: use <code>python3</code> . Windows: use <code>py -3</code> . Then rerun A3.
Something about failing to connect	The Ollama app is not running. Open it, wait for the icon, rerun.
model not found	The name in the file must match <code>ollama list</code> exactly. Open <code>1_agent_raw.py</code> in any text editor and change the <code>MODEL = ...</code> line near the top to your exact model name.
python or py not recognized	Python is not installed or not on your path. Use your partner's laptop today and install later.
It is taking a very long time	Press Control + C to stop it. Then check you are using the 0.5b model and not a larger one.
The answer contains JSON that looks like a tool call, but no tool ran	This is a real small-model failure, not your mistake. Write it down — it is a good thing to report in the discussion.